

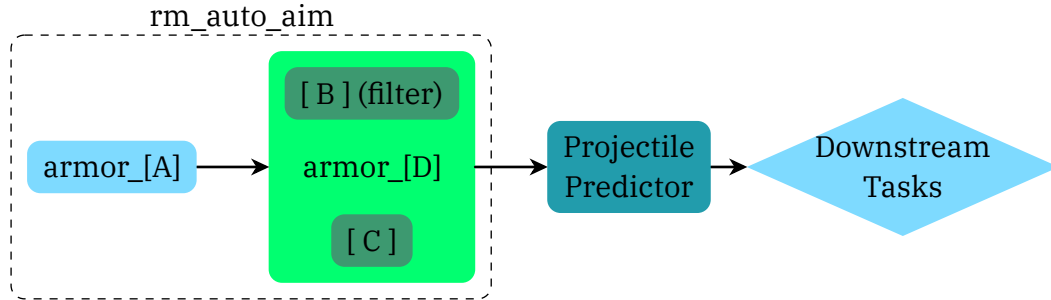
rm-vision Quiz

Author: [nice-mee](#)

概念

System Overview

根据 [rm_auto_aim](#) 完成以下图的填空



A: _____ B: _____ C: _____ D: _____

Questions

1. `armor_[D]` 分为两部分，请你分别简单解释这两部分的作用。
2. 在一个理想世界中，[B] 其实是可以不需要的，但在真实世界中如果去掉了 [B] 会出现什么问题？（其实这题和 P1 重复了😓，当做 P1 的 hint 就行）
3. `armor_[A]` 的作用是什么？
4. 整个系统最终输出的是什么？
5. `armor_[A]` 中将 2D 像素坐标转换为 3D 世界坐标的算法是什么？
6. `armor_[D]` 输出了什么？这些输出又是如何被 `Projectile Predictor` 利用的（非常 high-level 阐述即可）？

OpenCV 理论

请阅读 OpenCV 文档中 [Camera Calibration](#)（不需要了解 distortion 部分）和 [Pose Estimation](#)¹

如果看不懂矩阵乘法，只要知道矩阵只是一个表达线性系统的记号：

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{bmatrix} \iff \begin{cases} ax + by + cz = \alpha \\ dx + ey + fz = \beta \\ gx + hy + iz = \gamma \\ jx + ky + lz = \delta \end{cases}$$

文档里的矩阵式子都可以用等式组的方式写出来，只是矩阵表达更加简洁。

Problem 1

假设相机坐标系与世界坐标系重合，即 $R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, $t = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$ 。经过标定已知 $f_x = 0.09$, $f_y = 0.11$, $c_x = 0.001$, $c_y = 0$ 。

1. 请你求出一个世界坐标系下 $P_w = \begin{bmatrix} 10 \\ 5 \\ 20 \end{bmatrix}$ 的物体在相机上最终成像的 2D 坐标 $\begin{bmatrix} u \\ v \end{bmatrix}$ 。
2. 假如 $R = \begin{bmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}$ （绕 Z 轴旋转 45° ）， t 不变，结果又是多少？

Problem 2

也许你已经发现了，[Camera Calibration](#) 出现了对原有坐标系升维的操作。

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

这实际上是把欧几里得空间的坐标映射到了仿射空间的坐标（齐次坐标）。通过这种升维的方式，我们能把平移向量 t 优雅的放入矩阵，让旋转+平移重新变为矩阵·向量乘法。你可以尝试只在三维的情况下表示旋转+平移，你会发现是无法直接用矩阵·向量乘法直接表达的：

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

我们只能使用矩阵·向量乘法再加上一个向量的方式表达，这显然并不是我们想要的。

为什么我们如此关心矩阵？这是因为当零散的式子被放入矩阵后，我们可以做许多更高级的数学操作（如果还用曾经的方式表达会繁琐无比，几乎看不懂）；从计算机的角度来说，矩阵乘法运算可以用 GPU, AVX 一类的加速器运算，效率会比直接一行行硬算高很多。即使都是纯 CPU 计算，矩阵乘法也可以用 locality / 汇编一类的 trick 加速很多，Eigen 库里面的矩阵乘法比手写快非常多。

在上面的式子中我们已经发现了旋转+平移变换（刚体变换）：

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

¹macOS 用户用 Chromium-based 浏览器打开，Safari 有渲染 bug

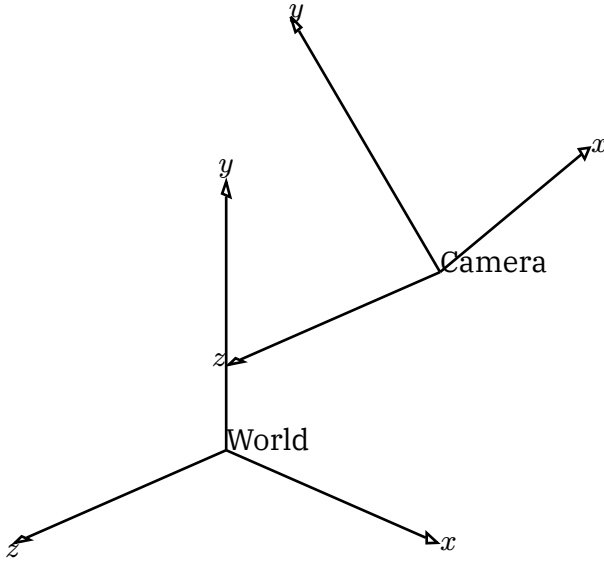
问题： 相似变换由均匀缩放、旋转和平移三个操作构成（注意顺序），缩放时坐标会围绕原点放大 s 倍。请你推出仿射空间下相似变换的矩阵。

Problem 3 (Optional)

我们继续讨论世界坐标系到相机坐标系的刚体变换，其表达为：

$$\begin{bmatrix} P_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} P_w \\ 1 \end{bmatrix} \iff \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

假设有以下场景：



以世界坐标系为中心，我们可以观察到世界坐标系与相机坐标系的坐标轴向量分别为

$$x_w = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, y_w = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, z_w = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, x_c = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \\ 0 \end{bmatrix}, y_c = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \\ 0 \end{bmatrix}, z_c = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

我们还能观察到在世界坐标系下，相机坐标系的原点坐标为 $\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$

请你求出世界坐标系到相机坐标系刚体变换的 R 与 t (Hint: 这题几乎不需要任何计算，但很容易得出错误结论)

另辟蹊径 (Optional)

在组会上我简单提及了 armor_detector 的算法：

- 通过 `cv::threshold` 过滤出亮度较高的部分
- 利用 `cv::findContours` 找出其中疑似多边形的图案，通过一些规则找出可能为灯条的多边形
- 对所有可能的灯条进行两两配对，符合规则的灯条对会被识别为装甲板

这套算法只用了 OpenCV 提供的传统 CV 方法，但其实还存在一些其他依赖 NN (Neural Network) 的做法：

- 传统/ NN 混合：先用 NN 找出所有疑似是装甲板的区域，在这些区域中找出灯条的坐标
- 纯 NN 方法：直接用 NN 输出灯条的四个点

你认为这些方案与 armor_detector 的传统方案之间的 tradeoff（优缺点）是什么呢？这是一个 open question，可以随便说说看法 😊